Ministerul Educaţiei şi Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică şi Microelectronică

Raport
la
Lucrarea de laborator Nr. 5

Disciplina: Tehnici de programare

Tema: ”Python pe web: importuri, metode și funcții”

A efectuat : st. gr.SI - 261 Vizir]Maximilian

A verificat : asistent universitar Dumitrascu Marius

Chișinău - 2026

SARCINA LUCRĂRII

1. Rezolvarea celor 50 de exerciţii din cinci părţi: importuri, metode şi atribute, funcţii proprii, module din biblioteca standard, propriul modul.

2. Lucrul cu requests, urllib, re, html.parser, hashlib, json, socket şi ssl, pe site-ul cybercor.org.

3. Crearea modulului webtools.py şi a programului main.py, care generează raportul site-ului în report.json.

4. Exerciţiul bonus: rescrierea funcţiilor get_title() şi extract_links() cu biblioteca BeautifulSoup.

MEDIUL DE LUCRU

Lucrarea a fost realizată în Python 3.14, într-un mediu virtual. Biblioteca externă requests a fost instalată cu pip install requests, iar pentru exercițiul bonus a fost adăugată biblioteca beautifulsoup4. Toate cererile folosesc timeout, iar între cererile repetate către cybercor.org există time.sleep(1).

Constantele comune ale fișierelor:

```
# Laborator: funcții, metode și importuri pe web
# Student: Vizir Maximilian

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde
```

DESFĂȘURAREA LUCRĂRII

Partea I. Importuri (exerciţiile 1-8)

Exerciţiul 1. Primul import

S-a importat modulul requests și s-a afișat versiunea lui, apoi s-a importat urllib.request. În comentariu s-a explicat că requests este o bibliotecă externă, care necesită pip install, iar urllib face parte din biblioteca standard.

```
# Exercitiul 1
import requests
#print(requests.__version__)

import urllib.request
print(urllib.request.__version__)

    # requests necesita pip install, deoarece este un modul care
    # este scris de o alta persoana si publicata,
    # iar urllib se instaleaza impreuna cu instalarea la python
```

Exerciţiul 2. Două stiluri de import

Pagina principală a fost descărcată cu requests.get() în două stiluri: import requests și from requests import get. Comentariul compară cele două variante: prefixul evită conflictele de nume, iar varianta scurtă reduce codul.

```
# Exercitiul 2
page2 = requests.get(BASE_URL, timeout=TIMEOUT)
print(page2.status_code)
time.sleep(1)
from requests import get
page2_ = get(BASE_URL, timeout=TIMEOUT)
print(page2_.status_code)

    #avantajul la import requests, este ca daca vom avea doua metode cu aceeasi denumire
    #programul va intelege la care ne adresam deoarece le chemam prin .
    #in al doilea caz, chemarea este mai rapida, adresandu-ne doar prin get, dar pot
    #aparea conflicte daca sunt mai multe metode la fel
```

Exerciţiul 3. Alias-uri

Modulul a fost importat cu alias (import requests as rq) și folosit pentru a descărca pagina principală. Comentariul explică când un alias ajută (convenții cunoscute) și când încurcă (nume alese la întâmplare).

```
# Exercitiul 3
import requests as rq
time.sleep(1)
```

```
page3 = rq.get(BASE_URL, timeout=TIMEOUT)
print(page3.status_code)

    #un alias face codul mai usor atunci cand este importat corect. spre exemplu a fost
    #decis pe internet sau ca o conventie ca requests sa fie importat ca rq, astfel
denumirea
    #e usoara de tinut minte. Dar poate sa faca mai greu de citit, in caz ca ar fi
importat ca
    #niste denumiri aleatorii, spre exemplu import requests as x sau y sau ceva. atunci
este mai
    #greu de citit y.get(...)
```

Exerciţiul 4. Doar biblioteca standard

Pagina a fost descărcată doar cu urllib.request.urlopen(), afişându-se .status şi primele 200 de caractere din corp. Pe cybercor.org cererea cu urllib a returnat 403, de aceea exerciţiul a fost rulat pe ECHO_URL.

```
# Exercitiul 4
time.sleep(1)
page4 = urllib.request.urlopen(ECHO_URL)
print(page4.status)
print(page4.read().decode("utf-8")[:200])

    #pagina cyberorg returneaza 403 cand incerci cu urllib.requests sa o descarci
```

Exerciţiul 5. Priviţi în interiorul unui modul

Cu dir(requests) au fost analizate trei nume din modul, iar în comentariu s-a precizat dacă fiecare este funcţie, clasă sau modul.

```
#Exercitiul 5
print(dir(requests))

     #Requests este o clasa, in genere este o conventie care spune ca clasele se denumesc
doar cu litere mari
     #get este o functie, deoarece aceasta ne returneaza o valoare, care este pagina pe
care o descarcam
     #status_code este o metoda deoarece iti da codul de stare al paginii dintr-o
variabila deja creata
```

Exercițiul 6. Citiți documentația

help(requests.get) nu listează direct parametrul timeout: el este documentat în requests.request și ajunge la get() prin **kwargs. Parametrul a fost folosit în toate cererile.

```
#Exercitiul 6
help(requests.get)
help(requests.request)

     #in requests.get nu este parametrul necesar timeout, el se gaseste in
requests.request, de asemenea
     #a fost folosit in exercitiile anterioare ca argument timeout=TIMEOUT
```

Exercițiul 7. Cronometrarea unei cereri

Durata unei cereri a fost măsurată cu time.perf_counter() și comparată cu response.elapsed. Valorile sunt apropiate, diferența fiind de ordinul milisecundelor.

```
#Exercitiul 7
time.sleep(1)
start_time = time.perf_counter()
page7 = requests.get(BASE_URL, timeout=TIMEOUT)
end_time = time.perf_counter()
print(page7.status_code)
print(page7.elapsed)
print(end_time - start_time)

     # la afisarea ambelor timpuri din modulul time si modulul requests, se afiseaza
aproximativ acelasi timp
     #diferenta este de milisecunde
```

Exercițiul 8. Când un import eșuează

Scriptul încearcă import bs4 într-un bloc try / except ImportError. Modulul nu era instalat, așa că s-a afișat mesajul cu instrucțiunea pip install beautifulsoup4.

```
#Exercitiul 8
try:
    import bs4
except ImportError:
    print("Instalați modulul cu: pip install beautifulsoup4.")

     # aici se afiseaza la rulare print din exceptie, deoarece modulul nu este instalat.
```

Rezultatul rulării: part1_imports.py

```
.env ❯ python3 part1_imports.py

Exercitiul 1
2.34.2
3.14

Exercitiul 2
200
200

Exercitiul 3
200

Exercitiul 4
200
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <title>httpbin.org</title>
    <link href="https://fonts.googleapis.com/css?family=Open+Sans:400,700|Source+Code+Pro:300,600|Tit

Exercitiul 5
['ConnectTimeout', 'ConnectionError', 'DependencyWarning', 'FileModeWarning', 'HTTPError', 'JSONDecodeError', 'NullHandler', 'PreparedRequest
', 'ReadTimeout', 'Request', 'RequestException', 'RequestsDependencyWarning', 'Response', 'Session', 'Timeout', 'TooManyRedirects', 'URLRequi
red', '__all__', '__author__', '__author_email__', '__build__', '__builtins__', '__cached__', '__cake__', '__copyright__', '__description__',
 '__doc__', '__file__', '__license__', '__loader__', '__name__', '__package__', '__path__', '__spec__', '__title__', '__url__', '__version__'
, '_check_cryptography', '_internal_utils', '_types', 'adapters', 'annotations', 'api', 'auth', 'certs', 'chardet_version', 'charset_normaliz
er_version', 'check_compatibility', 'codes', 'compat', 'cookies', 'delete', 'exceptions', 'get', 'head', 'hooks', 'logging', 'models', 'optio
ns', 'packages', 'patch', 'post', 'put', 'request', 'session', 'sessions', 'ssl', 'status_codes', 'structures', 'urllib3', 'utils', 'warnings
']

Exercitiul 6
Help on function get in module requests.api:

get(
    url: _t.UriType,
    params: _t.ParamsType = None,
    **kwargs: Unpack[_t.GetKwargs]
) -> Response
    Sends a GET request.

    :param url: URL for the new :class:`Request` object.
    :param params: (optional) Dictionary, list of tuples or bytes to send
        in the query string for the :class:`Request`.
    :param \*\*kwargs: Optional arguments that ``request`` takes.
    :return: :class:`Response <Response>` object
    :rtype: requests.Response
```

Fig.1.1 Rezultatul rulării part1_imports.py

Partea II. Metode şi atribute ale obiectelor web (exerciţiile 9-20)

Exerciţiul 9. Elementele de bază ale răspunsului

Au fost afişate atributele status_code, ok, url şi encoding ale răspunsului. Toate sunt atribute (se accesează fără paranteze rotunde), pentru că returnează informaţii deja stocate în obiect.

```
#Exercitiul 9
import time
print("\nExercitiul 9")
import requests
page9 = requests.get(BASE_URL, timeout=TIMEOUT)
print("Status_code " , page9.status_code)
print("Is OK? " , page9.ok)
print("URL: " , page9.url)
print("Codarea: " , page9.encoding)

     # toate acesteasunt atribute, deoarece returneaza o informatie despre pagina
noastra, se cheama fara paranteze rotunde
```

Exerciţiul 10. Tratarea erorilor cu o metodă

raise_for_status() pe pagina principală nu face nimic când răspunsul este 200. Pentru o pagină inexistentă, cybercor.org returnează tot succes, de aceea excepţia requests.HTTPError a fost demonstrată pe ECHO_URL.

```
#Exercitiul 10
print("\nExercitiul 10")
try:
    time.sleep(1)
    page10 = requests.get(ECHO_URL + "/this-page-does-not-exist-so-do-you",
timeout=TIMEOUT)
    page10.raise_for_status()
except requests.HTTPError:
    print("Pagina care ati incercat-o nu exista")

     #in cazul dat BASE_URL + string-ul oricum ne returneaza succes, de aceea am incercat
cu ECHO_URL
     #pentru ca sa-mi afiseze print-ul din exceptie
```

Exerciţiul 11. Toate antetele

Au fost afişate toate antetele răspunsului, în forma Nume: valoare, parcurgând response.headers.items().

```
#Exercitiul 11
print("\nExercitiul 11")
for c,k in page9.headers.items():
    print(c + " : " + k)
```

Exerciţiul 12. Metode de dicţionar

Cu headers.get() s-au afișat antetele Server și Content-Type, cu valoarea implicită „lipseste”. Content-Type și content-type dau același rezultat, deoarece response.headers nu ține cont de litere mari și mici.

```
#Exercitiul 12
print("\nExercitiul 12")
#page12 = requests.get(BASE_URL, timeout=TIMEOUT)
print(page9.headers.get("Server", "lipseste"))
print(page9.headers.get("Content-Type", "lipseste"))
print(page9.headers.get("content-type"))
     #aici ori Content-Type ori content-type, ne returneaza aceeasi valoare, deoarece nu
este case-sensitive
     #iar lipseste nu se afiseaza, deoarece valorile server si content-type sunt gasite
```

Exerciţiul 13. Înlănţuirea metodelor de şir

S-a numărat apariția cuvântului „cyber” cu response.text.lower().count(). Metodele se pot înlănțui pentru că lower() returnează un șir nou, pe care se apelează count().

```
# Exercitiul 13
print("\nExercitiul 13")
#page13 = requests.get(BASE_URL, timeout=TIMEOUT)
print(page9.text.lower().count("cyber"))

     #putem sa le inlantuim deoarece lower() returneaza un sir, si deja asupra acestui
sir chemam count()
```

Exerciţiul 14. Găsirea titlului

Titlul paginii a fost extras cu find() și feliere (slicing) între <title> și </title>, apoi curățat cu strip().

```
#Exercitiul 14
print("\nExercitiul 14")
#page14 = requests.get(BASE_URL, timeout=TIMEOUT)
start = page9.text.find("<title>") + len("<title>")
end = page9.text.find("</title>")
print(page9.text[start:end].strip())
```

Exercițiul 15. Numărarea liniilor

Codul HTML a fost împărțit în linii cu splitlines(); s-au afișat numărul de linii și lungimea celei mai lungi linii, cu max(lines, key=len).

```
#Exercitiul 15
print("\nExercitiul 15")
#page15 = requests.get(BASE_URL, timeout=TIMEOUT)
lines = page9.text.splitlines()
print(len(lines))
print(len(max(lines, key=len)))
```

Exercițiul 16. Verificarea HTTPS

Cu startswith() s-a verificat dacă adresa finală începe cu https și s-a afișat mesajul „Conexiune securizata” sau „Conexiune nesecurizata”.

```
#Exercitiul 16
print("\nExercitiul 16")
startwith = page9.url.startswith(("https"))
if startwith:
    print("Conexiune securizata")
else:
    print("Conexiune nesecurizata")

time.sleep(1)
```

Exercițiul 17. Urmărirea redirecționărilor

Cererea către http://cybercor.org a fost urmărită prin response.history: pentru fiecare redirecționare s-au afișat codul de stare (301) și URL-ul, apoi URL-ul final.

```
#Exercitiul 17
print("\nExercitiul 17")
page17 = requests.get("http://cybercor.org", timeout=TIMEOUT)
for res in page17.history:
     print("URL initial",res.url)
     print("Codul de stare: ",res.status_code)
print("URL final: ",page17.url, "\nCodul de stare: ",page17.status_code)

time.sleep(1)
```

Exerciţiul 18. HEAD versus GET

S-au comparat requests.head() şi requests.get(): len(content) este 1540 la GET şi 0 la HEAD, deoarece HEAD returnează doar antetele, fără corpul paginii.

```
#Exercitiul 18
print("\nExercitiul 18")
page18_get = requests.get(BASE_URL, timeout=TIMEOUT)
time.sleep(1)
page18_head = requests.head(BASE_URL, timeout=TIMEOUT)
print(len(page18_get.content))
print(len(page18_head.content))

     #la executare, lungimea la get este de 1540, dar la head este 0, deoarece cu get
primesti si corpul la html
     #dar cu head primesti doar niste date, care le afisezi cu .status_code sau .headers
```

Exerciţiul 19. Cookie-uri

S-au parcurs cookie-urile din response.cookies, afişând atributele name şi secure; dacă nu există niciun cookie, se afişează un mesaj.

```
#Exercitiul 19
print("\nExercitiul 19")
if len(page9.cookies) == 0:
     print("Nici un cookie gasit")
else:
     for c in page9.cookies:
          print(c.name)
          print(c.secure)
```

Exerciţiul 20. Sesiuni

S-a creat o sesiune requests.Session() cu un User-Agent personalizat, iar cererea către ECHO_URL/headers a confirmat în răspuns că antetul a fost trimis.

```
#Exercitiul 20
print("\nExercitiul 20")
sesiune = requests.Session()
sesiune.headers.update({"User-Agent": "WebLab-Numele Meu"})
time.sleep(1)
page20 = sesiune.get(ECHO_URL +"/headers", timeout=TIMEOUT)
print(page20.json()["headers"]["User-Agent"])
```

Rezultatul rulării: part2_imports.py

```
.env ❯ python3 part2_imports.py

Exercitiul 9
Status_code  200
Is OK?  True
URL:  https://cybercor.org/
Codarea:  utf-8

Exercitiul 10
Pagina care ati incercat-o nu exista

Exercitiul 11
Date : Tue, 20 Oct 2026 08:57:25 GMT
Content-Type : text/html; charset=utf-8
Transfer-Encoding : chunked
Connection : keep-alive
Nel : {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
Cache-Control : no-cache
Report-To : {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=6TXLJmiw%2FCPocVOiX2nL2VGaMaMAIF
LXFvvWVOSmFrtiCl82DHvlwIEQE4EfZQvkLbNSLa2x7Shdre4AzqUoSFVs7G7Hn%2B4z8UiT75K65PJhZ6Gga%2B4Rt0Yb7LEu30s%3D"}]}
Last-Modified : Wed, 14 Oct 2026 19:03:10 GMT
X-Powered-By : Express
X-Content-Type-Options : nosniff
X-Frame-Options : DENY
X-XSS-Protection : 1; mode=block
Referrer-Policy : strict-origin-when-cross-origin
cf-cache-status : DYNAMIC
Content-Encoding : zstd
Server : cloudflare
CF-RAY : a4d6d09cfb20c9ed-OTP
alt-svc : h3=":443"; ma=86400

Exercitiul 12
cloudflare
text/html; charset=utf-8
text/html; charset=utf-8

Exercitiul 13
9

Exercitiul 14
CYBERCOR

Exercitiul 15
32
218

Exercitiul 16
Conexiune securizata

Exercitiul 17
URL initial http://cybercor.org/
Codul de stare:  301
URL final:  https://cybercor.org/
Codul de stare:  200

Exercitiul 18
1540
0

Exercitiul 19
Nici un cookie gasit

Exercitiul 20
WebLab-Numele Meu
```

Fig.2.1 Rezultatul rulării part2_imports.py

Partea III. Scrierea propriilor funcţii (exerciţiile 21-34)

Exerciţiul 21. Prima funcţie

Funcţia fetch(url) returnează obiectul răspuns; a fost apelată pentru BASE_URL și s-a afișat codul de stare.

```
#Exercitiul 21
import requests, time
print("Exercitiul 21")
def fetch(url):
     """Descarcă pagina și returnează obiectul răspuns."""
     response = requests.get(url, timeout=TIMEOUT)
     return response

print(fetch(BASE_URL).status_code)
```

Exerciţiul 22. Returnarea unei singure valori

get_status(url) returnează doar codul de stare. A fost apelată pentru /, /robots.txt și /sitemap.xml, cu time.sleep(1) între apeluri.

```
#Exercitiul 22
print("\nExercitiul 22")
def get_status(url : str) -> int:
     """Returnează codul de stare al unui URL."""
     response = requests.get(url, timeout=TIMEOUT)
     return response.status_code
time.sleep(1)
print(get_status(BASE_URL+"/"))
time.sleep(1)
print(get_status(BASE_URL+"/robots.txt"))
time.sleep(1)
print(get_status(BASE_URL+"/itemap.xml"))
```

Exerciţiul 23. Parametri impliciţi

Funcția a primit parametrul implicit timeout=10 și a fost apelată o dată cu valoarea implicită și o dată cu timeout=3.

```
#Exercitiul 23
print("\nExercitiul 23")
def fetch_23(url, timeout=10):
      """Descarcă pagina, cu timeout configurabil (implicit 10 secunde)."""
      response = requests.get(url, timeout=timeout)
      return response
time.sleep(1)
print(fetch_23(BASE_URL, timeout=3))
time.sleep(1)
print(fetch_23(BASE_URL))
```

Exercițiul 24. O singură sarcină pe funcție

get_title() primește pagina descărcată și returnează titlul, reutilizând logica de la exercițiul 14. Funcția nu face operații de rețea: descărcarea se face separat.

```
#Exercitiul 24
print("\nExercitiul 24")
def get_title(html):
      """Returnează titlul paginii din răspunsul primit."""
      start = html.text.find("<title>") + len("<title>")
      end = html.text.find("</title>")
      return html.text[start:end].strip()
time.sleep(1)
page24 = requests.get(BASE_URL, timeout=TIMEOUT)
print(get_title(page24))
```

Exercițiul 25. Docstring-uri

Funcției get_title i s-a adăugat un docstring, afișat cu help(get_title).

```
#Exercitiul 25
print("\nExercitiul 25")
print(help(get_title))
```

Exercițiul 26. Adnotări de tip

S-au adăugat adnotări de tip funcțiilor. Apelul get_status(123) a generat o eroare din requests (MissingSchema): adnotările nu opresc programul, ele sunt doar indicii pentru cel care citește codul.

```
#Exercitiul 26
print("\nExercitiul 26")
try:
      print(get_status(123))
except requests.exceptions.MissingSchema:
      print("Functia get_status primeste ca valoare pagina, nu int")

      #adnotarile de tip sunt doar pentru persoana ce citeste codul, de aceea oricum
returneaza eroare la chemarea functiei
      #cu int in loc de cea ceruta str
```

Exercițiul 27. Fără opriri neașteptate

page_exists(url) returnează True sau False; excepția requests.RequestException este prinsă, deci funcția nu se oprește la o eroare de rețea.

```
#Exercitiul 27
print("\nExercitiul 27")
def page_exists(url):
      """Returnează True dacă pagina poate fi descărcată, altfel False."""
      try:
            response = requests.get(url, timeout=TIMEOUT)
            return True
      except requests.RequestException:
            return False
time.sleep(1)
print(page_exists(BASE_URL))
```

Exercițiul 28. Returnarea unui dicționar

check_paths(base, paths) returnează un dicționar {cale: cod de stare}, cu pauză de o secundă între cereri.

```
#Exercitiul 28
print("\nExercitiul 28")
path = ["/", "/robots.txt", "/sitemap.xml"]
def check_paths(base, paths):
      """Returnează un dicționar {cale: cod de stare} pentru fiecare cale."""
      status_codes = {}
      for p in paths:
            response = requests.get(base + p, timeout=TIMEOUT)
            status_codes[p] = response.status_code
            time.sleep(1)
      return status_codes
time.sleep(1)
print(check_paths(BASE_URL, path))
```

Exercițiul 29. Argumente cu nume

get_header(url, name, default) returnează valoarea unui singur antet; a fost apelată cu argument cu nume, name="Server".

```
#Exercitiul 29
print("\nExercitiul 29")
def get_header(url, name, default="lipseste"):
      """Returnează valoarea unui antet sau valoarea implicită."""
      response = requests.get(url, timeout=TIMEOUT)
      return response.headers.get(name, default)
print(get_header(BASE_URL, "Server"))
```

Exerciţiul 30. Antete de securitate

security_headers(url) verifică prezența a cinci antete de securitate şi returnează un dicţionar {antet: True/False}.

```
#Exercitiul 30
print("\nExercitiul 30")
def security_headers(url):
      """Verifică prezența a cinci antete de securitate; returnează {antet:
True/False}."""
      response = requests.get(url, timeout=TIMEOUT)
      exists = {}
      antet = ["Strict-Transport-Security", "Content-Security-Policy", "X-Frame-Options",
"X-Content-Type-Options", "Referrer-Policy"]
      for a in antet:
            exists[a] = a in response.headers
      return exists
time.sleep(1)
print(security_headers(BASE_URL))
```

Exerciţiul 31. Funcţii care apelează funcţii

score_headers(results) numără antetele prezente şi returnează un şir de forma „3/5”; funcția se apelează împreună cu security_headers.

```
#Exercitiul 31
print("\nExercitiul 31")
def score_headers(results):
      """Returnează scorul antetelor de securitate, de forma 3/5."""
      count = 0
      for k in results.values():
            if(k): count+=1
      return f"{count}/{len(results.values())}"
time.sleep(1)
print(score_headers(security_headers(BASE_URL)))
```

Exercițiul 32. Două funcții, o sarcină

fetch_robots(base) returnează textul fișierului robots.txt sau None, iar disallowed_paths(robots_text) returnează lista valorilor Disallow, inclusiv pentru intrarea None.

```
#Exercitiul 32
print("\nExercitiul 32")
def fetch_robots(base):
      """Returnează textul din robots.txt sau None dacă fișierul lipsește."""
      response = requests.get(base+"/robots.txt", timeout=TIMEOUT)
      if(response.status_code == 200):
            return response.text
      else: return None
def disallowed_paths(robots_text):
      """Returnează lista valorilor Disallow din robots.txt."""
      disallowed = []
      if(robots_text is None): return []
      text_lines = robots_text.splitlines()
      for t in text_lines:
            if(t.split(":",1)[0].strip().lower() == "disallow"):
                  disallowed.append(t.split(":",1)[1].strip())
      return disallowed
time.sleep(1)
print(disallowed_paths(fetch_robots(BASE_URL)))
print(disallowed_paths(None))
```

Exercițiul 33. Oricâte argumente

response_times(*urls) primește oricâte adrese și returnează un dicționar {url: secunde}, folosind response.elapsed.

```
#Exercitiul 33
print("\nExercitiul 33")
def response_times(*urls):
      """Returnează timpul de răspuns, în secunde, pentru fiecare URL."""
      times = {}
      for u in urls:
            response = requests.get(u,timeout=TIMEOUT)
            times[u] = response.elapsed.total_seconds()
            time.sleep(1)
      return times
time.sleep(1)
print(response_times(BASE_URL, ECHO_URL))
```

Exercițiul 34. Argumente cu nume colectate

log(message, **details) afișează mesajul urmat de fiecare detaliu primit prin argumente cu nume, în forma „mesaj | cheie=valoare”.

```
#Exercitiul 34
print("\nExercitiul 34")
def log(message, **details):
      """Returnează mesajul urmat de detalii, în forma mesaj | cheie=valoare."""
      mesaj = f"{message}"
      for c,k in details.items():
            mesaj += f" | {c}={k}"
      return mesaj
print(log("verificat", url=BASE_URL, status=200))
```

Rezultatul rulării: part3_imports.py

```
.env ❯ python3 part3_imports.py
Exercitiul 21
200

Exercitiul 22
200
200
200

Exercitiul 23
<Response [200]>
<Response [200]>

Exercitiul 24
CYBERCOR

Exercitiul 25
Help on function get_title in module __main__:

get_title(html)
    (Exercitiul 25: Returnează titlul paginii dintr-un șir HTML.

None

Exercitiul 26
Functia get_status primeste ca valoare pagina, nu int

Exercitiul 27
True

Exercitiul 28
{'/': 200, '/robots.txt': 200, '/sitemap.xml': 200}

Exercitiul 29
cloudflare

Exercitiul 30
{'Strict-Transport-Security': False, 'Content-Security-Policy': False, 'X-Frame-Options': True, 'X-Content-Type-Options': True, 'Referrer-Pol
icy': True}

Exercitiul 31
3/5

Exercitiul 32
[]
[]

Exercitiul 33
{'https://cybercor.org': 1.018264, 'https://httpbin.org': 2.06161}

Exercitiul 34
verificat | url=https://cybercor.org | status=200
```

Fig.3.1 Rezultatul rulării part3_imports.py

Partea IV. Alte module din biblioteca standard (exercițiile 35-43)

Exercițiul 35. Descompunerea unui URL

Un URL a fost descompus cu urllib.parse.urlparse(); s-au afișat scheme, netloc, path, query și fragment.

```
#Exercitiul 35
import time, requests, urllib, re
print("\nExercitiul 35")
def urlparser(url):
      """Afișează părțile unui URL: schemă, domeniu, cale, interogare, fragment."""
      parsed = urllib.parse.urlparse(url)
      print(parsed.scheme)
      print(parsed.netloc)
      print(parsed.path)
      print(parsed.query)
      print(parsed.fragment)
print(urlparser("https://cybercor.org/path?x=1#top"))
```

Exercițiul 36. Compunerea URL-urilor

Cu urljoin() legăturile relative /about, contact.html și ../index.html au fost transformate în URL-uri complete, pornind de la BASE_URL.

```
#Exercitiul 36
print("\nExercitiul 36")
def urljoiner(url):
      """Transformă legăturile relative în URL-uri complete, pornind de la URL-ul dat."""
      pagini = ["/about", "contact.html", "../index.html"]
      url_compus = []
      for p in pagini:
            url_compus.append(urllib.parse.urljoin(url,p))
      return url_compus
print(urljoiner(BASE_URL))
```

Exercițiul 37. Extragerea legăturilor

extract_links() extrage legăturile din HTML cu re.findall() și le returnează fără duplicate, păstrând ordinea.

```
#Exercitiul 37
print("\nExercitiul 37")
def extract_links(html):
      """Returnează legăturile din pagină, fără duplicate."""
      response = requests.get(html, timeout=TIMEOUT)
      links = re.findall(r'href="([^"]+)"', response.text)
      links_not_repeated = []
      for l in links:
            if (l not in links_not_repeated):
                  links_not_repeated.append(l)
      return links_not_repeated
time.sleep(1)
print(extract_links(BASE_URL))
```

Exercițiul 38. Interne sau externe?

split_links(links, domain) împarte legăturile în interne și externe: mai întâi urljoin(), apoi compararea urlparse().netloc cu domeniul.

```
#Exercitiul 38
print("\nExercitiul 38")
def split_links(links, domain):
      """Împarte legăturile în interne (același domeniu) și externe."""
      same = []
      different = []
      for l in links:
            full = urllib.parse.urljoin("https://"+domain,l)
            if(urllib.parse.urlparse(full).netloc == domain):
                  same.append(full)
            else:
                  different.append(full)
      return same, different
time.sleep(1)
print(split_links(extract_links(BASE_URL), urllib.parse.urlparse(BASE_URL).netloc))
```

Exercițiul 39. Propriile metode

Clasa ImageFinder moștenește HTMLParser și suprascrie handle_starttag() pentru a colecta atributul src al etichetelor <img>; a fost testată pe un HTML propriu și pe pagina reală.

```
#Exercitiul 39
print("\nExercitiul 39")
from html.parser import HTMLParser

class ImageFinder(HTMLParser):
    """Colectează atributul src al etichetelor <img> dintr-un HTML."""
    def __init__(self):
        """Pregătește lista goală în care se colectează imaginile."""
        super().__init__()
        self.images = []
```

```
    def handle_starttag(self, tag, attrs):
        """Adaugă în listă src-ul fiecărei etichete <img>."""
        if(tag == "img"):
              for attr in attrs:
                    if(attr[0] == "src"):
                          self.images.append(attr[1])
test_html = """
<html><body>
  <img src="/logo.png" alt="Logo">
  <IMG SRC="poza.jpg">
  <img alt="imagine fără src">
  <img src="https://cdn.example.com/banner.webp" />
  <a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

finder = ImageFinder()
finder.feed(test_html)
print(finder.images)

assert finder.images == [
    "/logo.png",                          # imagine obișnuită
    "poza.jpg",                           # tag scris cu majuscule
    "https://cdn.example.com/banner.webp",  # tag care se închide singur
], "Parserul nu a găsit exact imaginile așteptate"
print("Testul a trecut!")

print("\nIar acum pe pagina reala")
time.sleep(1)
page39 = requests.get(BASE_URL, timeout=TIMEOUT)
finder_real = ImageFinder()
finder_real.feed(page39.text)
print("verificare prin finder_real.images: ",len(finder_real.images), "imagini gasite")
for src in finder_real.images:
      print(src)
print("verificare prin count: ", page39.text.lower().count("<img"))
```

Exerciţiul 40. Amprenta paginii

page_fingerprint(url) returnează amprenta SHA-256 a paginii. Două rulări consecutive dau aceeași amprentă dacă pagina nu se schimbă; ar fi diferite dacă s-ar modifica măcar un caracter sau dacă pagina are conținut dinamic.

```
#Exercitiul 40
print("\nExercitiul 40")
import hashlib
def page_fingerprint(url):
      """Returnează amprenta SHA-256 a paginii."""
      response = requests.get(url, timeout=TIMEOUT)
      return hashlib.sha256(response.content).hexdigest()
time.sleep(1)
print(page_fingerprint(BASE_URL))
time.sleep(1)
```

```
print(page_fingerprint(BASE_URL))

      #s-ar schimba hash-ul in cazul in care codul html s-ar schimba macar putin (un
caracter)
      #s-au cand pagina are ceva dinamic,vreun contor sau ceva. atunci la fiecare
instalare a paginii
      #pot fi valori diferite, de aici si hash diferit.
```

Exercițiul 41. Salvarea în JSON

Antetele paginii au fost salvate în headers.json cu json.dump(), apoi încărcate înapoi cu json.load(), iar un antet a fost afișat din fișier.

```
#Exercitiul 41
print("\nExercitiul 41")
import json
with open("headers.json", "w") as file:
      json.dump(dict(page39.headers), file, indent=2)

with open("headers.json", "r") as file:
      headers = json.load(file)
print(headers.get("Server"))
```

Exercițiul 42. Interogare DNS

resolve(hostname) returnează adresa IP a domeniului cu socket.gethostbyname(); a fost afișată adresa IP pentru cybercor.org.

```
#Exercitiul 42
print("\nExercitiul 42")
import socket
def resolve(hostname):
      """Returnează adresa IP a unui nume de domeniu."""
      return socket.gethostbyname(hostname)
print(resolve(urllib.parse.urlparse(BASE_URL).netloc))
```

Exercițiul 43. Expirarea certificatului

cert_days_left(hostname) se conectează la portul 443, citește certificatul cu getpeercert() și returnează numărul de zile până la expirare.

```
#Exercitiul 43
print("\nExercitiul 43")
import ssl, socket
def cert_days_left(hostname):
      """Returnează zilele rămase până la expirarea certificatului."""
      context = ssl.create_default_context()
      with socket.create_connection((hostname,443)) as sock:
            with context.wrap_socket(sock, server_hostname=hostname) as ssock:
                  time_string = ssock.getpeercert()["notAfter"]
                  time_int = ssl.cert_time_to_seconds(time_string)
                  return (time_int-time.time())/86400
print(cert_days_left(urllib.parse.urlparse(BASE_URL).netloc))
```

Rezultatul rulării: part4_imports.py

```
.env › python3  part4_imports.py

Exercitiul 35
https
cybercor.org
/path
x=1
top
None

Exercitiul 36
['https://cybercor.org/about', 'https://cybercor.org/contact.html', 'https://cybercor.org/index.html']

Exercitiul 37
['/assets/index-CErP0EKt.css']

Exercitiul 38
(['https://cybercor.org/assets/index-CErP0EKt.css'], [])

Exercitiul 39
['/logo.png', 'poza.jpg', 'https://cdn.example.com/banner.webp']
Testul a trecut!

Iar acum pe pagina reala
verificare prin finder_real.images:  0 imagini gasite
verificare prin count:  0

Exercitiul 40
7eb6b3f9bc4a2568acb90498a5012d282f109974b6660503d2d164e4c451d292
7eb6b3f9bc4a2568acb90498a5012d282f109974b6660503d2d164e4c451d292

Exercitiul 41
cloudflare

Exercitiul 42
104.21.91.71

Exercitiul 43
44.3206037432728
```

Fig.4.1 Rezultatul rulării part4_imports.py

Partea V. Construirea propriului modul (exerciţiile 44-50 şi bonus)

Exerciţiul 44. Crearea unui modul

Funcţiile fetch, get_status, get_title şi security_headers au fost mutate în modulul webtools.py şi apelate din main.py prin import webtools.

```
# ---- webtools.py ----
#Exercitiul 44, adaugarea functiilor intr-un fisier .py aparte
TIMEOUT = 10
DEFAULT_HEADERS = {"User-Agent" : "WebLab-<Numele vostru :D>"}
import requests,time,re, json, socket, ssl,urllib.parse

def fetch(url):
      """Descarcă pagina și returnează obiectul răspuns."""
      response = requests.get(url, headers=DEFAULT_HEADERS ,timeout=TIMEOUT)
      return response

def get_status(html):
      """Returnează codul de stare al unui răspuns."""
      return html.status_code

def get_title(html):
      """Returnează titlul paginii din răspunsul primit."""
      start = html.text.find("<title>") + len("<title>")
      end = html.text.find("</title>")
      return html.text[start:end].strip()

def security_headers(html):
    """Verifică prezența a cinci antete de securitate; returnează {antet: True/False}."""
    antete = ["Strict-Transport-Security", "Content-Security-Policy",
              "X-Frame-Options", "X-Content-Type-Options", "Referrer-Policy"]
    exists = {}
    for a in antete:
        exists[a] = a in html.headers
    return exists

# ---- main.py ----
#Exercitiul 44
import time
print("\nExercitiul 44")
import webtools
print(webtools.get_title(webtools.fetch(BASE_URL)))
```

Exerciţiul 45. Importarea anumitor nume

În main.py importul a trecut la from webtools import get_title, security_headers. Dacă main.py ar avea și o funcție proprie get_title, numele ar indica doar funcția definită sau importată ultima, fără nicio eroare.

```
# ---- main.py ----
#Exercitiul 45
print("\nExercitiul 45")
from webtools import fetch, get_title, security_headers
time.sleep(1)
```

```
print(get_title(fetch(BASE_URL)))

      #in caz ca am avea o functie get_title in main.pyб atunci va depinde de ce a fost
ultima
      #from ... import ... sau def...() . Daca ultima este importul, atunci scriind
get_title
      #se va chema din import, daca functia, atunci se va chema functia
```

Exercițiul 46. Blocul main

La finalul lui webtools.py s-a adăugat blocul if __name__ == "__main__". Autotestul rulează doar la python webtools.py, deoarece la import __name__ este „webtools”, nu „__main__”.

```
# ---- webtools.py ----
#Exercitiul 46
if __name__ == "__main__":
     print("\nExercitiul 46")
     print("Autotest:", get_status(fetch("https://cybercor.org")))

     #autotest ruleaza doar in caz ca o chemam prin python3 webtools.py, nu si prin
python3 main.py
     #deoarece python in primul caz ii da valoarea de __main__ la variabila __name__, iar
in al doilea
     #caz, ii da valoarea numelui modulului
```

Exercițiul 47. Constante de modul

În webtools.py s-a adăugat constanta DEFAULT_HEADERS cu un User-Agent propriu, folosită în fetch(); în main.py ea este afișată cu print(webtools.DEFAULT_HEADERS).

```
# ---- webtools.py ----
DEFAULT_HEADERS = {"User-Agent" : "WebLab-<Numele vostru :D>"}

def fetch(url):
      """Descarcă pagina și returnează obiectul răspuns."""
      response = requests.get(url, headers=DEFAULT_HEADERS ,timeout=TIMEOUT)
      return response

# ---- main.py ----
#Exercitiul 47
print("\nExercitiul 47")
print(webtools.DEFAULT_HEADERS)
```

Exerciţiul 48. Argumente din linia de comandă

Cu argparse, main.py primeşte adresa site-ului din linia de comandă (python main.py https://cybercor.org) şi o transmite funcţiilor, în locul lui BASE_URL.

```
# ---- main.py ----
#Exercitiul 48
print("\nExercitiul 48")
import argparse
parser = argparse.ArgumentParser(description="Hmmm")
parser.add_argument("url", help="adresa site-ului")
args = parser.parse_args()
print(webtools.get_title(webtools.fetch(args.url)))
time.sleep(1)
```

Exerciţiul 49. Un raport CSV

Rezultatul funcţiei check_paths() a fost salvat în raport.csv, cu coloanele path, status şi checked_at (ora, cu datetime.now().isoformat()).

```
# ---- webtools.py ----
def check_paths(base, path):
      """Returnează codul de stare al unei căi de pe site."""
      response = requests.get(base+path, timeout=TIMEOUT)
      return response.status_code

# ---- main.py ----
#Exercitiul 49
print("\nExercitiul 49")
import csv,time
from datetime import datetime
path = ["/", "/robots.txt", "/sitemap.xml"]
check_time = []
status_code = []
for p in path:
      status_code.append(webtools.check_paths(BASE_URL,p))
      check_time.append(datetime.now().isoformat())
      time.sleep(1)
with open("raport.csv", "w", newline="") as file:
      writer = csv.writer(file)
      writer.writerow(["path","status","checked_at"])
      for i in range(3):
            writer.writerow([path[i],status_code[i],check_time[i]])
```

Exerciţiul 50. Proiect final: raportul site-ului

Funcţia site_report(url) din webtools.py colectează informaţiile cerute (cod de stare, titlu, adresă IP, redirecţionări, scor de securitate, zile până la expirarea certificatului, legături interne şi externe, căi interzise) şi le salvează în report.json.

```
# ---- webtools.py ----
def site_report(url):
      """Colectează informații despre un site și le salvează în report.json."""
      hostname = urllib.parse.urlparse(url).netloc
```

```
      page = fetch(url)
      status = get_status(page)
      title = get_title(page)
      score = score_headers(security_headers(page))
      same, different = split_links(extract_links(page), hostname)

      ip = resolve(hostname)
      days_left = cert_days_left(hostname)

      time.sleep(1)
      page_http = fetch("http://" + hostname)
      chain = []
      for res in page_http.history:
            chain.append(f"{res.status_code} -> {res.headers.get('Location')}")

      time.sleep(1)
      disallowed = disallowed_paths(fetch_robots(url))

      report = {
            "url": url,
            "status": status,
            "final_url": page.url,
            "title": title,
            "ip": ip,
            "redirects": chain,
            "security_score": score,
            "cert_days_left": days_left,
            "internal_links": len(same),
            "external_links": len(different),
            "disallowed_paths": disallowed,
      }

      with open("report.json", "w") as file:
            json.dump(report, file, indent=2, ensure_ascii=False)
      return report


# ---- main.py ----
#Exercitiul 50
print("\nExercitiul 50\nScriere in fisier")
time.sleep(1)
webtools.site_report(BASE_URL)
```

Bonus. BeautifulSoup

get_title() și extract_links() au fost rescrise cu BeautifulSoup: codul este mai scurt, iar parserul tratează corect cazuri la care găsirea de text sau expresia regulată eșuează (<TITLE lang="ro">, href cu ghilimele simple).

```
# ---- main.py ----
#Exercitiul bonus
from bs4 import BeautifulSoup

def get_title_bs(html):
      """Returnează titlul paginii, folosind BeautifulSoup."""
      soup = BeautifulSoup(html.text, "html.parser")
      if(soup.title is None or soup.title.string is None): return ""
      return soup.title.string.strip()

def extract_links_bs(html):
      """Returnează legăturile din pagină, fără duplicate, folosind BeautifulSoup."""
      soup = BeautifulSoup(html.text, "html.parser")
      links = []
      for tag in soup.find_all(href=True):
            if(tag["href"] not in links):
                  links.append(tag["href"])
      return links

      # Cu BeautifulSoup codul e mai scurt (extract_links nu mai are nevoie de re)
      # și mai sigur: find("<title>") și regex-ul href="..." ratează <TITLE lang="ro">,
      # href='...' sau href = "...", pe când BeautifulSoup înțelege structura HTML-ului.
      # Dezavantaj: e o bibliotecă externă, care cere pip install.
```

Rezultatul rulării: python main.py https://cybercor.org (la anexa sunt atasate continuturile fisierelor, Anexa C)

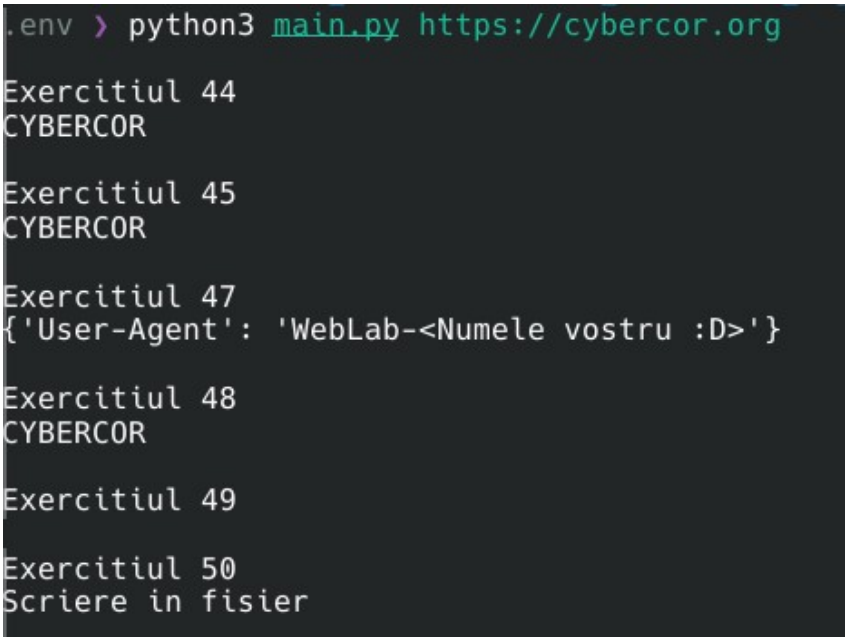


Fig.5.1 Rezultatul rulării python main.py https://cybercor.org

CONCLUZIE

În cadrul acestei lucrări de laborator am parcurs cele cinci părți ale temei: importuri, metode și atribute ale obiectelor web, funcții proprii, module din biblioteca standard și crearea propriului modul.

Am lucrat cu requests, urllib, re, html.parser, hashlib, json, socket, ssl, csv și argparse, iar la final am adunat funcțiile în modulul webtools.py și am creat raportul site-ului în report.json. Exercițiul bonus cu BeautifulSoup a arătat că un parser HTML face codul mai scurt și mai sigur decât căutarea de text.

BIBLIOGRAFIE

https://docs.python.org/3/library/
https://httpbin.org/
https://cybercor.org/

ANEXA A. Structura directorului web_lab

Fișierele predate în arhiva web_lab_<numele>.zip:

Fișier	Conținut
`part1_imports.py`	Partea I, exercițiile 1-8
`part2_imports.py`	Partea II, exercițiile 9-20
`part3_imports.py`	Partea III, exercițiile 21-34
`part4_imports.py`	Partea IV, exercițiile 35-43
`webtools.py`	Modulul propriu: funcțiile din exercițiile 44-50, site_report() și bonusul
`main.py`	Programul principal, rulat cu python main.py https://cybercor.org
`raport.csv`	Rezultatul funcției check_paths() (exercițiul 49)
`report.json`	Raportul site-ului, generat de site_report() (exercițiul 50)
`headers.json`	Antetele paginii principale (exercițiul 41)

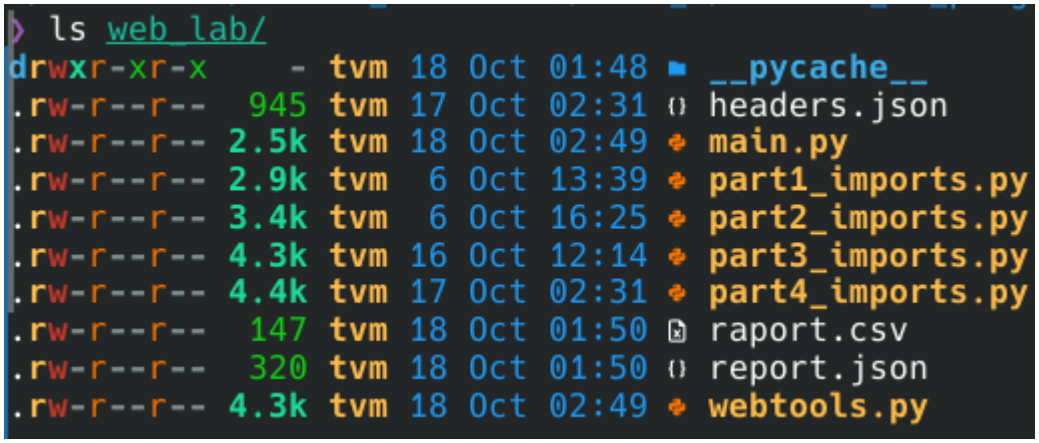


Fig.A.1 Conținutul directorului web_lab

ANEXA B. Fișiere de date generate

B.1 raport.csv

```
path,status,checked_at
/,200,2026-10-17T17:42:06.868350
/robots.txt,200,2026-10-17T17:42:07.961085
/sitemap.xml,200,2026-10-17T17:42:09.093282
```

B.2 report.json

```
{
 "url": "https://cybercor.org",
 "status": 200,
 "final_url": "https://cybercor.org/",
 "title": "CYBERCOR",
 "ip": "172.67.211.96",
 "redirects": [
   "301 -> https://cybercor.org/"
 ],
 "security_score": "3/5",
 "cert_days_left": 46,
 "internal_links": 1,
 "external_links": 0,
 "disallowed_paths": []
}
```

B.3 headers.json

```
{
 "Date": "Fri, 16 Oct 2026 23:31:28 GMT",
 "Content-Type": "text/html; charset=utf-8",
 "Transfer-Encoding": "chunked",
 "Connection": "keep-alive",
 "Nel": "{\"report_to\":\"cf-nel\",
\"success_fraction\":0.0,\"max_age\":604800}",
 "Cache-Control": "no-cache",
 "Report-To": "{\"group\":\"cf-nel\",\"max_age\":604800,\"endpoints\":[{\"url\":
\"https://a.nel.cloudflare.com/report/v4?s=stH6esgHoltrAoR%2
B1DbgtXR4MS6SPy%2BnpQE5ikwWzyFWua7LnYGDq6NvIRkIWJ8PVqiE0pWFTY4VjnvucVg3bRW8wNh8a
iRAgcURwrCknBiziAl%2Fba3r9Jug%2B2ygJzA%3D\"}]}",
 "Last-Modified": "Wed, 14 Oct 2026 19:03:10 GMT",
 "X-Powered-By": "Express",
 "X-Content-Type-Options": "nosniff",
 "X-Frame-Options": "DENY",
```

```
 "X-XSS-Protection": "1; mode=block",
 "Referrer-Policy": "strict-origin-when-cross-origin",
 "cf-cache-status": "DYNAMIC",
 "Content-Encoding": "zstd",
 "Server": "cloudflare",
 "CF-RAY": "a4badb75fa0e5e34-KIV",
 "alt-svc": "h3=\":443\"; ma=86400"
}
```